



siSS

Web Application & API Penetration Test Report

May 06, 2026



for checking i used this

The conclusions and recommendations in this report represent the opinions of Securify. Determinations of appropriate corrective action(s) are the responsibility of the entity receiving the report. This report and any other materials furnished by Securify in connection with this engagement are confidential and may not be duplicated, modified, or otherwise reproduced and distributed without the express prior written consent of Securify or Client Name.

Table of Contents

Table of Contents	3
Introduction	4
Approach	5
Runtime Application Vulnerability Assessment	5
Scope	6
Application Details	6
User Roles (Web application & API)	6
Tools	6
Assessment Limitation	7
Findings and Recommendation	8
Risk Classification	8
Measurement of Impact	8
Measurement of Likelihood	9
Overall Risk	10
Zero-risk Issues	10
Vulnerabilities	11
Summary	11
Detailed Vulnerabilities	12
Privilege Escalation	12
Account Takeover via Host Header Injection	15
Open Redirection	18
Cross-origin Resource Sharing	20
Weak Password Policy in Use	22
Appendix A	24

Introduction

As part of an ongoing security program, siss identified the need to conduct an application security assessment of its Web application & APIs.

The following lists the objectives of this assessment:

- Determine the overall security posture of the application
- Provide a list of key findings and recommendations for remediation
- Document the assessment scope, risk evaluation, and final outcomes

This report includes the following parameters and results of the assessment:

Approach

Securify performed a Runtime Application Vulnerability Assessment of the client environment using a combination of manual testing, guided analysis, and targeted verification of identified attack paths.

Runtime Application Vulnerability Assessment

The runtime assessment focused on security behavior observable in the live application and API flows, including authentication, authorization, input handling, business logic, and sensitive data exposure.

- Information Gathering - The application was reviewed as an anonymous, authenticated, and privileged user to understand differences in access and behavior.
- Authentication Testing - Authentication mechanisms were evaluated to determine the strength of login controls, password policies, and account recovery processes.
- Authorization Testing - Tests were conducted to identify weaknesses in access control, including attempts to access other users' data or privileged functionality.
- Session Management - Session handling was assessed to ensure secure creation, storage, and invalidation of session tokens.
- Input Validation Attacks - User-controlled input fields were tested with malformed and malicious data to identify injection flaws and logic bypasses.
- Business Logic Testing - The application workflows were reviewed to identify opportunities to misuse or bypass intended processes.

Scope

The assessment was conducted between Start Date and End Date. The scope of the assessment was limited to the environments and targets below:

The scope of the assessment was limited to the environments and targets below:

Application Details

Name	URL
Application Name 1	http://test.com
Application Name 2	http://admin.test.com

User Roles (Web application & API)

Role	Description
Customer	Authenticated customer user
Admin	Privileged administrative user

Tools

Tool Name	Description
Burp Professional Pro	An advanced proxy for testing web security.
OpenSSL	An open-source package to assess security in transit.
Nmap	A tool used to discover hosts and services on a network.
SQLMap	Python-based CLI tool that identifies SQL injection issues.
Acunetix Pro	An automated scanner to perform authenticated scans on the web app / APIs.
cURL Utility	Command line utility to perform HTTP/s requests.

The following components and tests were out of scope for this review:

- Any applications and infrastructure external to the siss's Web Application & API.
- In cases where the siss's Web Application & API had inbound and/or outbound interfaces with.
- Another application, the interfaces, and communications were considered in scope. All other external elements were excluded.
- Supporting policies, procedures, and processes.
- Social Engineering.
- Software Development Life Cycle.

Assessment Limitation

This assessment was performed within the agreed timebox and only against systems explicitly included in scope. Absence of identified vulnerabilities should not be interpreted as a guarantee that no issues remain.

Findings and Recommendation

Risk Classification

Risk is determined by evaluating the combined effect of impact and likelihood for each issue identified during testing.

Risk Matrix

Impact x Likelihood

Medium	High	Critical
Low	Medium	High
Low	Low	Medium

Measurement of Impact

Factor	Description	Effect on Impact
Classification of Data Loss	If an attacker exploited this vulnerability, what type of data would they access or steal? Non-sensitive, non-customer-specific data would lower the impact.	Less critical data reduces the overall impact.
Data Integrity	Could the attacker modify or delete data they shouldn't? If yes, what kind of data could they corrupt?	Non-critical or limited modification rights reduce the impact of exploitation.
Blast Radius	How many users or systems would be affected by this vulnerability? For instance, an obscure service affects fewer users, while a public system affects many.	A small number of victims lowers the severity.

Impact reflects the potential business, technical, operational, and data-related consequences if a vulnerability is successfully exploited.

- Critical Impact: Severe compromise of confidentiality, integrity, or availability with material business or operational consequence.
- High Impact: Significant unauthorized access, data exposure, or business-process abuse affecting sensitive functions or users.
- Medium Impact: Meaningful security weakness with constrained scope, partial exposure, or limited operational effect.
- Low Impact: Minor weakness with limited practical consequence or low-value exposure.

Measurement of Likelihood

Factor	Description	Effect on Likelihood
Required Skill Level of Attacker	How technically skilled does the attacker need to be to exploit the vulnerability? The higher the required skills, the lower the likelihood.	Complex skill requirements lower the chances of exploitation.
Number of Potential Attackers	How many attackers could potentially exploit this vulnerability? If access to the system is limited, fewer attackers can exploit it.	Fewer attackers reduce the probability of exploitation.
Cost of Exploit	How expensive are the tools or resources required for exploitation? High costs reduce accessibility for attackers.	Expensive tools lower the likelihood of exploitation.
Ease of Exploit	How simple is it for the attacker to exploit the vulnerability once discovered? Does it rely on external events or tools?	Complex or multi-step scenarios reduce the chances of successful exploitation.

Likelihood reflects how feasible exploitation is in practice, considering attacker capability, required access, and environmental constraints.

- High Likelihood: Exploitation is straightforward and requires limited skill, access, or environmental dependency.
- Medium Likelihood: Exploitation is feasible but requires some knowledge, sequencing, or favorable conditions.
- Low Likelihood: Exploitation requires significant effort, elevated preconditions, or specialized capability.

Overall Risk

Overall risk is derived from the intersection of impact and likelihood and is used to prioritize remediation efforts.

Zero-risk Issues

Items categorized as zero-risk or informational are included where useful for context but should not be treated as exploitable vulnerabilities.

Vulnerabilities

Summary

Below is a summary of the vulnerabilities discovered during the assessment, ranked in order of risk as determined.

Vulnerability	Risk
Unauthenticated Access to Sensitive Auction Bid Information	Critical
Missing Rate Limit on OTP Generation Allows Flooding	High
OTP Brute-Force Vulnerability on Email/Phone Change Functionality	Medium

Detailed Vulnerabilities

Unauthenticated Access to Sensitive Auction Bid Information

Risk: Critical

Description:

A business logic error was identified in the auction bidder portal, allowing unauthenticated access to sensitive bidding information. The application exposes bid details, including current bid values and maximum bid values, through API endpoints that lack proper authentication and authorization controls. An attacker can manipulate request parameters to retrieve this sensitive data for live auction items without needing to authenticate to the system.

Affected URL:

Impact and Likelihood:

https://www.alpha.auctionninja.com/auctionninjademo/readbidvalue_product_257.php?pid=<product-id>&auctionid=<auction-id>&uid=<user-id>&ItemNo=<lot number>,
https://www.alpha.auctionninja.com/auctionninjademo/readbidvalue_account.php?pid=<product id>&auctionid=<auction-id>&uid=<user-id>

- Impact:

The disclosure of maximum bid values is highly sensitive in an auction system. An attacker or a malicious seller can leverage this information to manipulate auction prices, artificially inflate bids, or gain an unfair financial advantage. This undermines the integrity and fairness of the auction process, potentially leading to significant financial losses for legitimate bidders and reputational damage for the auction platform.

- Likelihood:

The vulnerability is highly likely to be exploited as it involves direct access to API endpoints without any authentication or authorization checks. An attacker can easily craft requests with valid product and auction IDs to retrieve sensitive bidding information, making the exploitation straightforward and requiring minimal technical sophistication.

Steps to Reproduce:

Step 1: Identify a live auction product ID, auction ID, and user ID (if applicable) for a target item.

Step 2: Construct a GET request to the vulnerable endpoint, for example:

`https://www.alpha.auctionninja.com/auctionninjademo/readbidvalue\_product\_257.php?pid=<product-id>&auctionid=<auction-id>&uid=<user-id>&ItemNo=<lot number>`.

Step 3: Send the request without any authentication headers or session cookies.

Step 4: Observe the response, which will contain sensitive bidding information, including the current bid value and potentially the maximum bid value placed by a bidder.

Recommendations:

Authentication: Implement robust authentication mechanisms for all API endpoints that handle or expose sensitive information. Ensure that only authenticated and authorized users can access bid details.

Authorization: Enforce strict authorization checks to ensure that users can only access bid information relevant to their own account or items they are authorized to view. Implement role-based access control (RBAC) where appropriate.

Secure API Design: Review and redesign API endpoints to ensure that sensitive data is not exposed without proper security controls. Avoid relying solely on client-side parameter manipulation for security.

Input Validation: Validate all input parameters to prevent manipulation attempts and ensure that only expected values are processed by the application logic.

References:

- <https://cwe.mitre.org/data/definitions/287.html>
- <https://cwe.mitre.org/data/definitions/863.html>
- <https://owasp.org/www-project-api-security/>

Missing Rate Limit on OTP Generation Allows Flooding

Risk: High

Description:

The application's endpoint responsible for changing mobile numbers, specifically 'editemail_ajax.php', lacks proper rate limiting. This vulnerability allows an attacker to repeatedly request One-Time Passwords (OTPs) to be sent to a new phone number without any restrictions, leading to an OTP flooding scenario.

Affected URL:

Impact and Likelihood:

https://www.alpha.auctionninja.com/editemail_ajax.php

- Impact:

The impact of OTP flooding can be significant. It can lead to denial of service for legitimate users by overwhelming their mobile devices with unwanted messages, potentially causing distress or preventing them from receiving critical communications. Furthermore, it can incur substantial SMS costs for the organization. In scenarios where OTPs are used for account recovery, this could be leveraged in social engineering attacks to disrupt user access or facilitate account takeover attempts if combined with other vulnerabilities.

- Likelihood:

The absence of rate limiting on the OTP generation functionality makes this vulnerability highly likely to be exploited. An attacker can easily automate requests to the affected endpoint, repeatedly triggering OTP delivery to a target phone number. This straightforward method of exploitation increases the feasibility and probability of a successful attack.

Steps to Reproduce:

Step 1: Log in to the application as a manager.

Step 2: Navigate to the functionality allowing a user to change their mobile number.

Step 3: Intercept the request made to 'https://www.alpha.auctionninja.com/editemail_ajax.php' when initiating a mobile number change.

Step 4: Modify the request to specify a new target phone number.

Step 5: Send the modified request repeatedly without any delay or rate limit enforcement.

Step 6: Observe that multiple OTPs are sent to the specified new phone number, confirming the lack of rate limiting.

Recommendations:

Implement Rate Limiting: Apply robust rate limiting mechanisms to the OTP generation endpoint ('editemail_ajax.php'). This should restrict the number of OTP requests from a single IP address, user account, or session within a defined time frame.

Introduce CAPTCHA/MFA: Integrate CAPTCHA or reCAPTCHA challenges before allowing OTP generation, especially after a few failed attempts or suspicious activity.

Session Management: Ensure that OTP generation requests are tied to a valid, authenticated user session and that session IDs are properly invalidated after logout or inactivity.

Logging and Monitoring: Implement comprehensive logging for all OTP generation attempts, including source IP, timestamp, and target phone number. Establish alerts for unusual patterns or high volumes of OTP requests to detect and respond to potential flooding attacks promptly.

References:

- https://owasp.org/www-project-web-security-testing-guide/v41/4-Authentication/OTG-AUTHN-007-Testing_for_Rate_Limiting
- <https://cwe.mitre.org/data/definitions/770.html>

OTP Brute-Force Vulnerability on Email/Phone Change Functionality

Risk: Medium

Description:

The application's functionality for changing a registered email address or phone number is vulnerable to brute-force attacks. When a user attempts to update their contact information, a 6-digit One-Time Password (OTP) is sent to the new email or phone for verification. However, the system lacks essential security controls to prevent or detect repeated failed OTP attempts. Specifically, there is no rate limiting on OTP submissions, no account lockout mechanism after multiple incorrect attempts, no CAPTCHA or similar verification, and the OTP remains valid for an unlimited number of attempts. This allows an attacker to systematically guess all 1,000,000 possible 6-digit combinations until the correct OTP is identified.

Affected URL:

Impact and Likelihood:

https://www.alpha.auctionninja.com/editemail_ajax.php,
https://www.alpha.auctionninja.com/editphone_code_ajax.php

- **Impact:**

The impact of this vulnerability is medium. A successful brute-force attack on the OTP verification allows an attacker to change the registered email address or phone number associated with a user's account. This could lead to unauthorized account modifications, potential account takeover if the email/phone is used for password recovery, or disruption of service by changing the legitimate user's contact details. While not directly leading to full account compromise without further steps, it significantly weakens the account's security posture and could be a precursor to more severe attacks.

- **Likelihood:**

The likelihood of exploitation is high due to the absence of any brute-force protection mechanisms. A 6-digit OTP provides 1,000,000 possible combinations, which can be easily exhausted by an automated script without any rate limiting or account lockout. The lack of an attempt counter or CAPTCHA further simplifies the attack, making it highly feasible for an attacker to successfully guess the correct OTP and gain unauthorized control over the associated email or phone verification.

Steps to Reproduce:

Step 1: Log in to the application as an authenticated user.

Step 2: Navigate to the profile settings or account management section where the email address or phone number can be changed.

Step 3: Initiate a request to change the email address or phone number to an arbitrary value.

Step 4: Observe that an OTP is sent to the provided email/phone for verification.

Step 5: Intercept the OTP verification request (e.g., to /editemail_ajax.php or /editphone_code_ajax.php).

Step 6: Send multiple requests with varying 6-digit OTPs to the verification endpoint.

Step 7: Confirm that the application does not implement rate limiting, account lockout, or CAPTCHA, allowing an attacker to brute-force all possible 6-digit OTPs until the correct one is found and the email/phone is successfully updated.

Recommendations:

Implement Rate Limiting: Introduce robust rate limiting on OTP verification attempts. For example, allow a maximum of 3-5 incorrect attempts within a short timeframe (e.g., 5 minutes) per user account and/or IP address.

Implement Account Lockout: After a predefined number of failed OTP attempts (e.g., 5 attempts), temporarily or permanently lock the account or the specific functionality (email/phone change) to prevent further brute-forcing.

Introduce CAPTCHA/MFA: Implement a CAPTCHA or other multi-factor authentication challenge (e.g., re-authentication with password) before allowing OTP submission or after a few failed attempts.

Invalidate OTP after Attempts: Ensure that an OTP becomes invalid after a certain number of failed attempts, even if the rate limit is not hit.

Increase OTP Complexity: Consider increasing the length and character set of the OTP to make it more resistant to brute-force attacks, though this should be combined with other controls.

Monitor and Alert: Implement logging and alerting for excessive failed OTP attempts to detect and respond to potential brute-force attacks in real-time.

References:

- https://owasp.org/www-project-top-10/2021/A07_2021-Identification_and_Authentication_Failures
- <https://cwe.mitre.org/data/definitions/307.html>

Appendix A

Penetration Test Scope	
V1	Authentication Verification Requirements
V1.1	Password Security Requirements
1.1.1	Verify that user-set passwords are at least 12 characters in length
1.1.2	Verify that passwords 64 characters or longer are permitted
1.1.3	Verify that passwords can contain spaces and that truncation is not performed. Consecutive multiple spaces MAY optionally be coalesced
1.1.4	Verify that Unicode characters are permitted in passwords. A single Unicode code point is considered a character, so 12 emoji or 64 kanji characters should be valid and permitted
1.1.5	Verify users can change their password
1.1.7	Verify that passwords submitted during account registration, login, and password change are checked against a set of breached passwords either locally (such as the top 1,000 or 10,000 most common passwords that match the system's password policy) or using an external API. If using an API a zero-knowledge proof or other mechanism should be used to ensure that the plain text password is not sent or used in verifying the breach status of the password. If the password is breached, the application must require the user to set a new non non-breached password
1.1.8	Verify that a password strength meter is provided to help users set a stronger password
1.1.9	Verify that there are no password composition rules limiting the type of characters permitted. There should be no requirement for upper or lower case or numbers or special characters
1.1.10	Verify that "paste" functionality, browser password helpers, and external password managers are permitted
1.1.11	Verify that the user can choose to either temporarily view the entire masked password, or temporarily view the last typed character of the password on platforms that do not have this as native functionality
V1.2	General Authenticator Requirements

1.2.1	Verify that anti-automation controls are effective at mitigating breached credential testing, brute force, and account lockout attacks. Such controls include blocking the most common breached passwords, soft lockouts, rate limiting, CAPTCHA, ever-increasing delays between attempts, IP address restrictions, or risk-based restrictions such as location, first login on a device, recent attempts to unlock the account, or similar. Verify that no more than 100 failed attempts per hour is possible on a single account
1.2.2	Verify that the use of weak authenticators (such as SMS and email) is limited to secondary verification and transaction approval and not as a replacement for more secure authentication methods. Verify that stronger methods are offered before weak methods, that users are aware of the risks, or that proper measures are in place to limit the risks of account compromise
1.2.3	Verify that secure notifications are sent to users after updates to authentication details, such as credential resets, email or address changes, and logging in from unknown or risky locations. The use of push notifications - rather than SMS or email - is preferred, but in the absence of push notifications, SMS or email is acceptable as long as no sensitive information is disclosed in the notification
1.2.4	Verify impersonation resistance against phishing, such as the use of multi-factor authentication, cryptographic devices with intent (such as connected keys with a push to authenticate), or at higher ALL levels, client-side certificates
1.2.5	Verify that where a credential service provider (CSP) and the application verifying authentication are separated, mutually authenticated TLS is in place between the two endpoints
V1.3	Credential Recovery Requirements
1.3.1	Verify that a system-generated initial activation or recovery secret is not sent in clear text to the user
1.3.2	Verify password hints or knowledge-based authentication (so-called "secret questions") are not present
1.3.3	Verify password credential recovery does not reveal the current password in any way
1.3.4	Verify shared or default accounts are not present (e.g. "root", "admin", or "sa").
1.3.5	Verify that if an authentication factor is changed or replaced, the user is notified of this event
1.3.6	Verify forgotten passwords, and other recovery paths using a secure recovery mechanism, such as TOTP or another soft token, mobile push, or another offline recovery mechanism

1.3.7	Verify that if OTP or multi-factor authentication factors are lost, that evidence of identity proofing is performed at the same level as during enrolment
V1.4	Single or Multi-Factor One Time Verifier Requirements
1.4.1	Verify that time-based OTPs have a defined lifetime before expiring
1.4.2	Verify that symmetric keys used to verify submitted OTPs are highly protected, such as by using a hardware security module or secure operating system-based key storage
1.4.3	Verify that time-based OTP can be used only once within the validity period
1.4.4	Verify that if a time-based multi-factor OTP token is re-used during the validity period, it is logged and rejected with secure notifications being sent to the holder of the device
V2	Session Management Verification Requirements
V2.1	Fundamental Session Management Requirements
2.1.1	Verify the application never reveals session tokens in URL parameters or error messages
2.1.1	Verify the application generates a new session token on user authentication
2.1.2	Verify that session tokens possess at least 64 bits of entropy
2.1.3	Verify the application only stores session tokens in the browser using secure methods such as appropriately secured cookies or HTML 5 session storage
V2.2	Session Logout and Timeout Requirements
2.2.1	Verify that logout and expiration invalidate the session token, such that the back button or a downstream relying party does not resume an authenticated session, including across relying parties
2.2.2	If authenticators permit users to remain logged in, verify that re-authentication occurs periodically both when actively used or after an idle period. (C6) 30 days 12 hours or 30 minutes of inactivity, 2FA optional 12 hours or 15 minutes of inactivity, with 2FA
2.2.3	Verify that the application terminates all other active sessions after a successful password change and that this is effective across the application, federated login (if present), and any relying parties
2.2.4	Verify that users are able to view and log out of any or all currently active sessions and devices
V2.3	Cookie-based Session Management
2.3.1	Verify that cookie-based session tokens have the 'Secure' attribute set

2.3.2	Verify that cookie-based session tokens have the 'HttpOnly' attribute set
2.3.3	Verify that cookie-based session tokens utilize the 'SameSite' attribute to limit exposure to cross-site request forgery attacks
2.3.4	Verify that cookie-based session tokens use "__Host-" prefix (see references) to provide session cookie confidentiality
2.3.5	Verify that if the application is published under a domain name with other applications that set or use session cookies that might override or disclose the session cookies, set the path attribute in cookie-based session tokens using the most precise path possible
V2.4	Token-based Session Management
2.4.2	Verify the application uses session tokens rather than static API secrets and keys, except with legacy implementations
2.4.3	Verify that stateless session tokens use digital signatures, encryption, and other countermeasures to protect against tampering, enveloping, replay, null cipher, and key substitution attacks
V3	Access Control Verification Requirements
V3.1	Operation Level Access Control
3.1.2	Verify that the application or framework enforces a strong anti-CSRF mechanism to protect authenticated functionality, and effective anti-automation or anti-CSRF protects unauthenticated functionality
V4	Validation, Sanitization, and Encoding Verification Requirements Control Objective
V4.1	Input Validation Requirements
4.1.1	Verify that the application has defenses against HTTP parameter pollution attacks, particularly if the application framework makes no distinction about the source of request parameters (GET, POST, cookies, headers, or environment variables)
4.1.2	Verify that frameworks protect against mass parameter assignment attacks, or that the application has countermeasures to protect against unsafe parameter assignment, such as marking fields private or similar
4.1.3	Verify that all input (HTML form fields, REST requests, URL parameters, HTTP headers, cookies, batch files, RSS feeds, etc) is validated using positive validation (whitelisting)
4.1.4	Verify that structured data is strongly typed and validated against a defined schema including allowed characters, length and pattern (e.g. credit card numbers or telephone, or validating that two related fields are reasonable, such as checking that suburb and zip/postcode match)

4.1.5	Verify that URL redirects and forwards only allow whitelisted destinations, or show a warning when redirecting to potentially untrusted content
4.1.6	Verify that all untrusted HTML input from WYSIWYG editors or similar is properly sanitized with an HTML sanitizer library or framework feature
4.1.7	Verify that unstructured data is sanitized to enforce safety measures such as allowed characters and length
4.1.8	Verify that the application sanitizes user input before passing to mail systems to protect against SMTP or IMAP injection
4.1.9	Verify that the application avoids the use of eval() or other dynamic code execution features. Where there is no alternative, any user input being included must be sanitized or sandboxed before being executed
4.1.10	Verify that the application protects against template injection attacks by ensuring that any user input being included is sanitized or sandboxed
4.1.11	Verify that the application protects against SSRF attacks, by validating or sanitizing untrusted data or HTTP file metadata, such as filenames and URL input fields, using whitelisting of protocols, domains, paths, and ports
4.1.12	Verify that the application sanitizes, disables, or sandboxes user-supplied SVG scriptable content, especially as they relate to XSS resulting from inline scripts, and foreign Objects
4.1.13	Verify that the application sanitizes, disables, or sandboxes user-supplied scriptable or expression template language content, such as Markdown, CSS or XSL stylesheets, BBCode, or similar
V4.2	Output encoding and Injection Prevention Requirements
4.2.1	Verify that output encoding is relevant for the interpreter and context required. For example, use encoders specifically for HTML values, HTML attributes, JavaScript, URL Parameters, HTTP headers, SMTP, and others as the context requires, especially from untrusted inputs (e.g. names with Unicode or apostrophes, such as ねこ or O'Hara).
4.2.2	Verify that output encoding preserves the user's chosen character set and locale, such that any Unicode character point is valid and safely handled
4.2.3	Verify that context-aware, preferably automated - or at worst, manual - output escaping protects against reflected, stored, and DOM-based XSS
4.2.4	Verify that data selection or database queries (e.g. SQL, HQL, ORM, NoSQL) use parameterized queries, ORMs, entity frameworks, or are otherwise protected from database injection attacks

4.2.5	Verify that where parameterized or safer mechanisms are not present, context-specific output encoding is used to protect against injection attacks, such as the use of SQL escaping to protect against SQL injection
4.2.6	Verify that the application protects against JavaScript or JSON injection attacks, including for eval attacks, remote JavaScript includes, CSP bypasses, DOM XSS, and JavaScript expression evaluation.
4.2.7	Verify that the application protects against LDAP Injection vulnerabilities, or that specific security controls to prevent LDAP Injection have been implemented
4.2.8	Verify that the application protects against OS command injection and that operating system calls use parameterized OS queries or use contextual command line output encoding
4.2.9	Verify that the application protects against Local File Inclusion (LFI) or Remote File Inclusion (RFI) attacks.
4.2.10	Verify that the application protects against XPath injection or XML injection attacks
V4.3	Deserialization Prevention Requirements
4.3.1	Verify that serialized objects use integrity checks or are encrypted to prevent hostile object creation or data tampering
4.3.2	Verify that the application correctly restricts XML parsers to only use the most restrictive configuration possible and to ensure that unsafe features such as resolving external entities are disabled to prevent testE
4.3.3	Verify that deserialization of untrusted data is avoided or is protected in both custom code and third-party libraries (such as JSON, XML and YAML parsers)
4.3.4	Verify that when parsing JSON in browsers or JavaScript-based backends, JSON.parse is used to parse the JSON document. Do not use eval() to parse JSON
V5	Error Handling and Logging Verification Requirements
V5.1	Error Handling
5.1.1	Verify that a generic message is shown when an unexpected or security sensitive error occurs, potentially with a unique ID which support personnel can use to investigate
V6	Data Protection Verification Requirements
V6.1	Sensitive Private Data
6.1.1	Verify that sensitive data is sent to the server in the HTTP message body or headers, and that query string parameters from any HTTP verb do not contain sensitive data
V7	Communications Verification Requirements

V7.1	Communications Security Requirements
7.1.1	Verify that secured TLS is used for all client connectivity, and does not fall back to insecure or unencrypted protocols.
7.1.2	Verify using online or up-to-date TLS testing tools that only strong algorithms, ciphers, and protocols are enabled, with the strongest algorithms and ciphers set as preferred
7.1.3	Verify that old versions of SSL and TLS protocols, algorithms, ciphers, and configurations are disabled, such as SSLv2, SSLv3, or TLS 1.0 and TLS 1.1. The latest version of TLS should be the preferred cipher suite
V8	Business Logic Verification Requirements
8.1.1	Verify the application will only process business logic flows for the same user in sequential step order and without skipping steps
8.1.2	Verify the application will only process business logic flows with all steps being processed in realistic human time, i.e. transactions are not submitted too quickly
8.1.3	Verify the application has appropriate limits for specific business actions or transactions which are correctly enforced on a per-user basis
8.1.4	Verify the application has sufficient anti-automation controls to detect and protect against data exfiltration, excessive business logic requests, excessive file uploads, or denial of service attacks
8.1.5	Verify the application has business logic limits or validation to protect against likely business risks or threats, identified using threat modeling or similar methodologies
8.1.6	Tested for IDOR's vulnerabilities which would allow other users to retrieve any kind of sensitive information like api key, tokens etc.
8.1.7	Verified Privilege Escalation vulnerabilities which could allow low privilege users to perform arbitrary operations leading potential account takeovers.
8.1.8	Verified Privilege Escalation vulnerabilities which could allow users to perform CRUD Operations on behalf of victim users.
8.1.9	Checked for Information Disclosure vulnerabilities via IDORs which could result in users PII Disclosure (PII).
8.1.10	Checked for Broken Access Control issues via browsed forcing.
8.1.11	Checked for Mass Assignment vulnerability which could allow attackers to gain extra privileges.
V9	File and Resources Verification Requirements

V9.1	File Upload Requirements
9.1.1	Verify that the application will not accept large files that could fill up storage or cause a denial of service attack
V9.2	File Integrity Requirements
9.2.1	Verify that files obtained from untrusted sources are validated to be of expected type based on the file's content
V10	API and Web Service Verification Requirements
V10.1	Generic Web Service Security Verification Requirements
10.1.1	Verify that all application components use the same encodings and parsers to avoid parsing attacks that exploit different URI or file parsing behavior that could be used in SSRF and RFI attacks
10.1.2	Verify that access to administration and management functions is limited to authorized administrators
10.1.3	Verify API URLs do not expose sensitive information, such as the API key, session tokens, etc
10.1.4	Verify that authorization decisions are made at both the URI, enforced by programmatic or declarative security at the controller or router, and at the resource level, enforced by model-based permissions
10.1.5	Verify that requests containing unexpected or missing content types are rejected with appropriate headers (HTTP response status 406 Unacceptable or 415 Unsupported Media Type)
V10.2	RESTful Web Service Verification Requirements
V11	Configuration Verification Requirements
V11.1	Dependency
11.1.1	Verify that all components are up to date
11.1.2	Verify that all unneeded features, documentation, samples, configurations are removed, such as sample applications, platform documentation, and default or example users
11.1.3	Verify that if application assets, such as JavaScript libraries, CSS stylesheets or web fonts, are hosted externally on a content delivery network (CDN) or external provider, Subresource Integrity (SRI) is used to validate the integrity of the asset
V11.2	Unintended Security Disclosure Requirements

11.2.1	Verify that web or application server and framework error messages are configured to deliver user-actionable, customized responses to eliminate any unintended security disclosures
11.2.2	Verify that web or application server and application framework debug modes are disabled in production to eliminate debug features, developer consoles, and unintended security disclosures
11.2.3	Verify that the HTTP headers or any part of the HTTP response do not expose detailed version information of system components
V11.3	HTTP Security Headers Requirements
11.3.1	Verify that every HTTP response contains a content type header specifying a safe character set (e.g., UTF-8, ISO 8859-1).
11.3.2	Verify that all API responses contain Content-Disposition: attachment; filename="api.json" (or other appropriate filename for the content type).
11.3.3	Verify that a content security policy (CSPv2) is in place that helps mitigate impact of XSS attacks like HTML, DOM, JSON, and JavaScript injection vulnerabilities
11.3.4	Verify that all responses contain X-Content-Type-Options: nosniff
11.3.5	Verify that HTTP Strict Transport Security headers are included on all responses and for all subdomains, such as Strict-Transport-Security: max-age=15724800; include subdomains
11.3.6	Verify that a suitable "Referrer-Policy" header is included, such as "no-referrer" or "same-origin"
11.3.7	Verify that a suitable X-Frame-Options or Content-Security-Policy: frame ancestors header is in use for sites where content should not be embedded in a third-party site
V11.4	Validate HTTP Request Header Requirements
11.4.1	Verify that the application server only accepts the HTTP methods in use by the application or API, including pre-flight OPTIONS
11.4.2	Verify that the supplied Origin header is not used for authentication or access control decisions, as the Origin header can easily be changed by an attacker
11.4.3	Verify that the cross-domain resource sharing (CORS) Access-Control-Allow-Origin header uses a strict white list of trusted domains to match against and does not support the "null" origin
11.4.4	Verify that HTTP headers added by a trusted proxy or SSO devices, such as a bearer token, are authenticated by the application